

Continuous Integration (CI) with GitHub Actions

A Comprehensive, Step-by-Step Tutorial

Using the Sakila Flask Application as a Real-World Example

Technologies Covered: Python, Flask, GitHub Actions, Docker, PyTest, Flake8

Table of Contents

- 1. Introduction to Continuous Integration (CI)**
 - 1.1 What is CI?**
 - 1.2 Why CI Matters**
 - 1.3 CI vs CD vs CD (Deployment vs Delivery)**
- 2. Introduction to GitHub Actions**
 - 2.1 What is GitHub Actions?**
 - 2.2 Key Concepts and Terminology**
 - 2.3 How GitHub Actions Works**
 - 2.4 Pricing and Usage Limits**
- 3. Prerequisites**
 - 3.1 Required Accounts and Tools**
 - 3.2 The Sakila Flask Application Overview**
- 4. Setting Up Your First GitHub Actions Workflow**
 - 4.1 Understanding the Directory Structure**
 - 4.2 Creating the Workflow File**
 - 4.3 Workflow File Anatomy — Line by Line**
 - 4.4 Pushing and Triggering Your First Workflow**
 - 4.5 Reading the Workflow Results**
- 5. Writing Tests for CI**
 - 5.1 Why Tests Are Essential for CI**
 - 5.2 Setting Up PyTest**
 - 5.3 Writing Unit Tests for the Flask App**
 - 5.4 Running Tests Locally**
- 6. Building a Complete CI Pipeline**
 - 6.1 Stage 1 — Code Linting with Flake8**
 - 6.2 Stage 2 — Running Automated Tests**
 - 6.3 Stage 3 — Building a Docker Image**
 - 6.4 The Complete Multi-Stage Workflow**
- 7. Managing Secrets and Environment Variables**

- 7.1** Why Never Hardcode Secrets
- 7.2** Adding Secrets in GitHub
- 7.3** Using Secrets in Workflows
- 8.** Working with Workflow Triggers
 - 8.1** Push and Pull Request Triggers
 - 8.2** Manual Triggers (workflow_dispatch)
 - 8.3** Scheduled Triggers (cron)
 - 8.4** Branch and Path Filters
- 9.** Using Pre-Built Actions from the Marketplace
 - 9.1** What Are Actions?
 - 9.2** Commonly Used Actions
 - 9.3** Pinning Action Versions
- 10.** Branch Protection and CI Enforcement
 - 10.1** Setting Up Branch Protection Rules
 - 10.2** Requiring Status Checks to Pass
- 11.** Adding a CI Status Badge to Your README
- 12.** Troubleshooting Common Issues
- 13.** Best Practices Summary
- 14.** Glossary of Terms

1. Introduction to Continuous Integration (CI)

1.1 What is CI?

Continuous Integration (CI) is a software development practice where developers frequently merge their code changes into a shared repository — typically multiple times a day. Each merge (or push) automatically triggers a series of automated processes that build the application, run tests, and perform quality checks.

The core idea is simple: instead of waiting until the end of a development cycle to integrate everyone's changes (which often leads to "integration hell"), you integrate continuously, catching bugs and conflicts early when they are small and easy to fix.

A typical CI workflow looks like this:

1. A developer writes code and commits it to a Git branch.
2. The developer pushes the branch to a remote repository (e.g., GitHub).
3. The CI system automatically detects the new push.
4. The CI system runs a predefined pipeline: install dependencies, lint code, run tests, build artifacts.
5. The CI system reports the results — pass or fail — back to the developer.
6. If the pipeline fails, the developer is notified and fixes the issue before merging.

1.2 Why CI Matters

CI is not just a nice-to-have — it is a fundamental practice in modern software development. Here is why:

Catch bugs early: Automated tests run on every push, so bugs are caught within minutes of being introduced — not weeks later during manual QA.

Reduce integration risk: Small, frequent integrations are far less risky than large, infrequent ones. Merge conflicts are smaller and easier to resolve.

Faster feedback loop: Developers get immediate feedback on whether their code works correctly with the rest of the codebase.

Consistent quality: Every change goes through the same automated checks — linting, testing, security scans — ensuring a baseline quality standard.

Confidence to refactor: With a good test suite running in CI, developers can refactor code knowing that CI will catch regressions.

Documentation of process: CI workflow files serve as living documentation of how the project is built, tested, and validated.

1.3 CI vs CD vs CD (Deployment vs Delivery)

These three terms are often mentioned together but mean different things:

Term	Full Name	Description
CI	Continuous Integration	Automatically build and test code on every push. Ensures code integrates correctly.
CD	Continuous Delivery	Extends CI — code is always in a deployable state. Deployment to production is a manual decision.
CD	Continuous Deployment	Extends Continuous Delivery — every change that passes CI is automatically deployed to production. No manual approval.

 **Note:** In this tutorial, we focus exclusively on CI — the automated build, lint, and test pipeline. Deployment topics (CD) are beyond our scope.

2. Introduction to GitHub Actions

2.1 What is GitHub Actions?

GitHub Actions is a CI/CD platform built directly into GitHub. It allows you to automate your software development workflows right from your GitHub repository. You can build, test, and deploy your code — all triggered by events in your repository such as pushes, pull requests, issue creation, and more.

Unlike external CI services (Jenkins, Travis CI, CircleCI), GitHub Actions is natively integrated with GitHub. There is no external service to set up, no webhooks to configure, and no separate dashboard to monitor. Everything lives inside your repository.

2.2 Key Concepts and Terminology

Before we begin, let us understand the key building blocks of GitHub Actions:

Workflow: A configurable automated process defined in a YAML file. A repository can have multiple workflows (e.g., one for CI, one for deployment). Workflow files are stored in the `.github/workflows/` directory.

Event: A specific activity that triggers a workflow. Common events include: `push` (code pushed to a branch), `pull_request` (a PR is opened or updated), `schedule` (cron-based timer), and `workflow_dispatch` (manual trigger).

Job: A set of steps that execute on the same runner (virtual machine). A workflow can have multiple jobs. By default, jobs run in parallel, but you can configure them to run sequentially using dependencies.

Step: An individual task within a job. A step can either run a shell command (using the `run` keyword) or use a pre-built Action (using the `uses` keyword). Steps within a job execute sequentially.

Action: A reusable, pre-packaged unit of work. Actions are the building blocks of steps. GitHub provides official actions (e.g., `actions/checkout`, `actions/setup-python`), and the community contributes thousands more via the GitHub Marketplace.

Runner: The virtual machine that executes your jobs. GitHub provides hosted runners with Ubuntu Linux, Windows, and macOS. You can also set up self-hosted runners on your own infrastructure.

Artifact: A file or collection of files produced during a workflow run (e.g., build outputs, test reports, Docker images). Artifacts can be shared between jobs or downloaded after the workflow completes.

2.3 How GitHub Actions Works

Here is the flow of a GitHub Actions CI pipeline:

7. You create a YAML workflow file in your repository under `.github/workflows/`.

8. You define what events trigger the workflow (e.g., push to main branch).
9. You define one or more jobs, each containing a series of steps.
10. When the triggering event occurs, GitHub automatically provisions a fresh virtual machine (runner).
11. The runner executes your jobs and steps in order.
12. Results (pass/fail) are displayed on the GitHub Actions tab and on pull requests.
13. If the workflow fails, you are notified via email (by default).

2.4 Pricing and Usage Limits

GitHub Actions is free for public repositories. For private repositories:

- Free tier: 2,000 minutes/month for GitHub Free accounts
- GitHub Pro: 3,000 minutes/month
- GitHub Team: 3,000 minutes/month
- GitHub Enterprise: 50,000 minutes/month

 **Note:** For students and educational use, GitHub provides free access through the GitHub Student Developer Pack, which includes additional Actions minutes.

3. Prerequisites

3.1 Required Accounts and Tools

Before starting this tutorial, make sure you have the following:

GitHub Account: Sign up at <https://github.com> if you don't have one. A free account is sufficient.

Git: Install Git on your local machine. Download from <https://git-scm.com/downloads>. Verify installation by running: `git --version`

Python 3.9+: Download from <https://python.org/downloads>. Verify with: `python --version`

pip: Python's package manager. Usually comes with Python. Verify with: `pip --version`

A Code Editor: VS Code (recommended), PyCharm, Sublime Text, or any editor you are comfortable with.

A GitHub Repository: You need a repository pushed to GitHub. We will use the Sakila Flask App as our example.

3.2 The Sakila Flask Application Overview

Throughout this tutorial, we use a real Flask web application — the Sakila DVD Rental Store app — as our example project. This application:

- Is built with Flask 2.3.3 (a Python web framework)
- Connects to a MySQL database using PyMySQL
- Provides CRUD operations for films, actors, customers, rentals, staff, inventory, and stores
- Includes reporting and CSV export features
- Has a Dockerfile for containerized deployment

The project structure looks like this:

```
sakila_flask_app/
├── .github/
│   └── workflows/          ← CI workflow files go here (we will create this)
├── static/
│   ├── script.js
│   └── style.css
├── templates/
│   ├── base.html
│   ├── dashboard.html
│   ├── films.html
│   └── ... (other templates)
└── tests/                  ← Test files go here (we will create this)
```



```
|   ├── __init__.py
|   └── test_app.py
└── app.py           ← Main Flask application
└── config.py        ← Database and app configuration
└── requirements.txt ← Python dependencies
└── Dockerfile       ← Docker image definition
└── README.md
```



```

    branches: [ main ]                # ...targeting the main branch

jobs:                                # Define the jobs
  build:                              # Job ID (can be any name)
    name: Build and Check             # Display name for the job
    runs-on: ubuntu-latest            # Runner OS (GitHub-hosted Ubuntu)

    steps:                            # Sequence of steps in this job
      - name: Checkout repository      # Step 1: Get the code
        uses: actions/checkout@v4     # Use the official checkout action

      - name: Set up Python             # Step 2: Install Python
        uses: actions/setup-python@v5 # Use the official Python setup action
        with:
          python-version: '3.9'        # Specify Python version

      - name: Install dependencies      # Step 3: Install packages
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Verify application import # Step 4: Check the app loads
        run: |
          python -c "from app import app; print('App imported successfully')"

```

4.3 Workflow File Anatomy — Line by Line

Let us break down every single line of this workflow file:

name: CI Pipeline

This sets the display name of the workflow. This name appears in the GitHub Actions tab of your repository. Choose a descriptive name that tells you what this workflow does.

on:

The "on" key defines which events trigger this workflow. Without this, the workflow would never run. We have configured two triggers:

- `push` → `branches: [ main ]` — The workflow runs whenever code is pushed to the main branch.
- `pull_request` → `branches: [ main ]` — The workflow runs whenever a pull request is opened or updated against the main branch.

jobs:

The "jobs" section contains all the jobs that this workflow will execute. Each job is identified by a unique key (in our case, "build"). A workflow can have multiple jobs.

runs-on: ubuntu-latest

This specifies which type of virtual machine (runner) the job will use. "ubuntu-latest" means the latest stable Ubuntu Linux runner provided by GitHub. Other options include: windows-latest, macos-latest.

steps: / uses: / run:

Steps are the individual tasks within a job. There are two types of steps:

- **uses:** — References a pre-built Action (e.g., actions/checkout@v4). These are reusable components from GitHub or the community.
- **run:** — Executes a shell command directly on the runner (e.g., pip install -r requirements.txt).

The pipe character (|) after "run:" allows multi-line commands.

4.4 Pushing and Triggering Your First Workflow

Now that you have created the workflow file, let us push it to GitHub and watch it run.

Step 3: Add, commit, and push the workflow file:

```
# Stage the new workflow file
git add .github/workflows/ci.yml

# Commit with a descriptive message
git commit -m "Add CI workflow with GitHub Actions"

# Push to the main branch
git push origin main
```

The moment you push, GitHub will detect the new workflow file and automatically start running it. Let us go see it in action.

4.5 Reading the Workflow Results

Step 4: Navigate to your repository on GitHub and follow these steps to view the results:

14. Go to your repository on GitHub (e.g., <https://github.com/your-username/your-repo>).
15. Click on the "Actions" tab at the top of the repository page.
16. You will see a list of workflow runs. Click on the most recent run (it should show "CI Pipeline").
17. Click on the job name ("Build and Check") to see the detailed step-by-step output.

18. Each step can be expanded to see its console output. Green checkmarks mean the step passed; red X marks mean it failed.

 **Note:** If you see a yellow dot, the workflow is still running. If you see a red X, click on the failed step to read the error message and fix the issue.

Understanding the status indicators:

Icon	Status	Meaning
 (Yellow dot)	In Progress	The workflow is currently running
 (Green check)	Success	All steps passed successfully
 (Red X)	Failure	One or more steps failed
 (Grey circle)	Skipped/Cancelled	The workflow was skipped or manually cancelled

5. Writing Tests for CI

5.1 Why Tests Are Essential for CI

A CI pipeline without tests is like a fire alarm without sensors — it exists, but it cannot actually detect problems. Tests are what give CI its power: they automatically verify that your code behaves correctly every time a change is made.

Without tests, your CI pipeline can only verify that:

- Dependencies install correctly
- The code has no syntax errors
- The application can import without crashing

With tests, your CI pipeline can verify that:

- Individual functions return correct results (unit tests)
- API endpoints return expected responses (integration tests)
- Business logic works correctly (functional tests)
- New changes do not break existing functionality (regression tests)

5.2 Setting Up PyTest

PyTest is the most popular testing framework for Python. It is simple, powerful, and has excellent integration with CI systems.

Step 1: Add testing dependencies to your requirements.txt:

```
# Add these lines to your requirements.txt
pytest==7.4.4
pytest-cov==4.1.0
flake8==6.1.0
```

What each package does:

- `pytest` — The testing framework itself. Discovers and runs test files automatically.
- `pytest-cov` — A pytest plugin that measures code coverage (what percentage of your code is tested).
- `flake8` — A code linting tool that checks your code for style issues and potential errors (we will use this in Chapter 6).

Step 2: Install the new dependencies locally:

```
pip install -r requirements.txt
```

5.3 Writing Unit Tests for the Flask App

Step 3: Create the test directory and files:

```
mkdir tests
```

Step 4: Create an empty `__init__.py` file in the tests directory. This tells Python that "tests" is a package:

```
# Create an empty file: tests/__init__.py
```

```
# On Linux/Mac:
```

```
touch tests/__init__.py
```

```
# On Windows:
```

```
type nul > tests\__init__.py
```

Step 5: Create the test file `tests/test_app.py` with the following content:

```
# tests/test_app.py
```

```
"""
```

```
Unit tests for the Sakila Flask Application.
```

```
These tests verify the application works correctly without  
requiring a database connection.
```

```
"""
```

```
import pytest
```

```
import sys
```

```
import os
```

```
# Add the project root to the Python path so we can import app
```

```
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__),  
'..')))
```

```
from app import app
```

```
@pytest.fixture
```

```
def client():
```

```
    """
```

```
    Create a test client for the Flask application.
```

A test client allows us to send HTTP requests to our app without actually running a server. This is how we test Flask routes.

```
"""
app.config['TESTING'] = True          # Enable testing mode
app.config['WTF_CSRF_ENABLED'] = False # Disable CSRF for testing
with app.test_client() as client:
    yield client
```

```
class TestAppConfiguration:
    """Test that the application is configured correctly."""

    def test_app_exists(self):
        """Test that the Flask app instance exists."""
        assert app is not None

    def test_app_is_flask_instance(self):
        """Test that 'app' is actually a Flask application."""
        from flask import Flask
        assert isinstance(app, Flask)

    def test_secret_key_is_set(self):
        """Test that a secret key is configured (required for
sessions/flash)."""
        assert app.config['SECRET_KEY'] is not None
        assert len(app.config['SECRET_KEY']) > 0

    def test_testing_mode(self, client):
        """Test that testing mode can be enabled."""
        assert app.config['TESTING'] is True
```

```
class TestRouteDefinitions:
    """
    Test that all expected routes are registered in the application.
    This does NOT test the route functionality (which needs a database),
    only that the routes exist.
    """
```



```

def test_dashboard_route_exists(self):
    """Test that the dashboard route '/' is registered."""
    rules = [rule.rule for rule in app.url_map.iter_rules()]
    assert '/' in rules

def test_films_route_exists(self):
    """Test that the films route is registered."""
    rules = [rule.rule for rule in app.url_map.iter_rules()]
    assert '/films' in rules

def test_actors_route_exists(self):
    """Test that the actors route is registered."""
    rules = [rule.rule for rule in app.url_map.iter_rules()]
    assert '/actors' in rules

def test_customers_route_exists(self):
    """Test that the customers route is registered."""
    rules = [rule.rule for rule in app.url_map.iter_rules()]
    assert '/customers' in rules

def test_rentals_route_exists(self):
    """Test that the rentals route is registered."""
    rules = [rule.rule for rule in app.url_map.iter_rules()]
    assert '/rentals' in rules

def test_reports_route_exists(self):
    """Test that the reports route is registered."""
    rules = [rule.rule for rule in app.url_map.iter_rules()]
    assert '/reports' in rules

def test_api_endpoints_exist(self):
    """Test that API endpoints are registered."""
    rules = [rule.rule for rule in app.url_map.iter_rules()]
    assert '/api/actor/<int:actor_id>' in rules
    assert '/api/film/<int:film_id>' in rules

```

```

class TestConfigModule:
    """Test the config.py module."""

    def test_config_import(self):
        """Test that the Config class can be imported."""
        from config import Config
        assert Config is not None

    def test_config_has_mysql_host(self):
        """Test that Config has MYSQL_HOST attribute."""
        from config import Config
        assert hasattr(Config, 'MYSQL_HOST')

    def test_config_has_mysql_db(self):
        """Test that Config has MYSQL_DB attribute."""
        from config import Config
        assert hasattr(Config, 'MYSQL_DB')

    def test_config_has_secret_key(self):
        """Test that Config has SECRET_KEY attribute."""
        from config import Config
        assert hasattr(Config, 'SECRET_KEY')

    def test_config_mysql_db_default(self):
        """Test that the default database name is 'sakila'."""
        from config import Config
        # Only test default if env var is not set
        if not os.environ.get('MYSQL_DB'):
            assert Config.MYSQL_DB == 'sakila'

```

Let us understand the key parts of this test file:

@pytest.fixture: A fixture is a function that provides test data or setup. Our "client" fixture creates a Flask test client that we can use to make HTTP requests without a real server.

Test Classes: We organize tests into classes (TestAppConfiguration, TestRouteDefinitions, TestConfigModule) to group related tests together. This makes test output easier to read.

assert statements: The "assert" keyword checks if a condition is True. If it is False, the test fails and pytest reports what went wrong.

 **Note:** These tests are designed to run without a MySQL database connection. This is important for CI because the GitHub Actions runner does not have our sakila database. In a production CI setup, you would configure a MySQL service container, but for learning purposes, we focus on tests that can run independently.

5.4 Running Tests Locally

Step 6: Before pushing, always run tests locally to make sure they pass:

```
# Run all tests
pytest tests/ -v

# Run tests with coverage report
pytest tests/ -v --cov=app --cov-report=term-missing
```

Understanding the output:

- The -v flag enables verbose mode — shows each test name and its result.
- The --cov flag generates a code coverage report showing which lines of code were executed during testing.
- PASSED means the test succeeded. FAILED means the assertion was false.

Example output:

```
tests/test_app.py::TestAppConfiguration::test_app_exists PASSED
tests/test_app.py::TestAppConfiguration::test_app_is_flask_instance PASSED
tests/test_app.py::TestAppConfiguration::test_secret_key_is_set PASSED
tests/test_app.py::TestRouteDefinitions::test_dashboard_route_exists PASSED
tests/test_app.py::TestRouteDefinitions::test_films_route_exists PASSED
...

===== 14 passed in 0.45s =====
```

6. Building a Complete CI Pipeline

Now that we have tests, let us build a proper multi-stage CI pipeline. A professional CI pipeline typically has multiple stages that run in sequence or parallel:

19. Lint — Check code quality and style
20. Test — Run automated tests
21. Build — Build the application (e.g., Docker image)

6.1 Stage 1 — Code Linting with Flake8

Linting is the process of analyzing your code for potential errors, style violations, and deviations from best practices. Flake8 is a popular Python linting tool that checks your code against PEP 8 (Python's official style guide).

Why lint in CI?

- Catches common mistakes: undefined variables, unused imports, syntax errors
- Enforces consistent code style across the team
- Prevents obvious bugs from being merged

Step 1: Create a Flake8 configuration file. Create a file named `.flake8` in your project root:

```
# .flake8
[flake8]
max-line-length = 120
exclude =
    .git,
    __pycache__,
    venv,
    env,
    .venv
ignore =
    E501,
    W503,
    W504
```

Configuration explained:

- `max-line-length = 120` — Allow lines up to 120 characters (default is 79, which is quite strict).
- `exclude` — Directories to skip during linting.
- `ignore` — Specific rules to ignore. E501 = line too long, W503/W504 = line break rules.

6.2 Stage 2 — Running Automated Tests

We already wrote our tests in Chapter 5. In our CI pipeline, we will run them using pytest and generate a coverage report. The coverage report tells us what percentage of our code is exercised by our tests.

6.3 Stage 3 — Building a Docker Image

Since our project includes a Dockerfile, we can add a step to verify that the Docker image builds successfully. This ensures that any changes to the code or dependencies do not break the Docker build process.

Our existing Dockerfile:

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

6.4 The Complete Multi-Stage Workflow

Step 2: Replace the content of `.github/workflows/ci.yml` with this complete pipeline:

```
# .github/workflows/ci.yml
# Complete CI Pipeline for Sakila Flask Application

name: CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  # =====
  # JOB 1: LINT — Check code quality
  # =====
  lint:
    name: Code Linting
    runs-on: ubuntu-latest
```

```

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: '3.9'

  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip
      pip install flake8

  - name: Run Flake8 linter
    run: |
      echo "Running Flake8 linting..."
      flake8 app.py config.py --count --show-source --statistics

# =====
# JOB 2: TEST – Run automated tests
# =====
test:
  name: Run Tests
  runs-on: ubuntu-latest
  needs: lint          # Only run tests if linting passes

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.9'

    - name: Cache pip dependencies

```

```

    uses: actions/cache@v4
    with:
      path: ~/.cache/pip
      key: ${ runner.os }}-pip-${ hashFiles('requirements.txt') }}
      restore-keys: |
        ${ runner.os }}-pip-

- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt
    pip install pytest pytest-cov

- name: Run tests with coverage
  run: |
    pytest tests/ -v --cov=app --cov-report=term-missing

# =====
# JOB 3: BUILD – Build Docker image
# =====
build:
  name: Build Docker Image
  runs-on: ubuntu-latest
  needs: test          # Only build if tests pass

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Build Docker image
    run: |
      echo "Building Docker image..."
      docker build -t sakila-app:${ github.sha }} .
      echo "Docker image built successfully!"

  - name: Verify Docker image
    run: |
      docker images sakila-app:${ github.sha }}

```

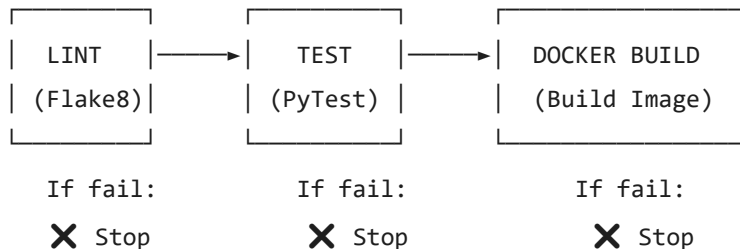
Let us understand the new elements in this workflow:

needs: lint / needs: test — The "needs" keyword creates dependencies between jobs. This means: Test job only runs if Lint passes, and Build job only runs if Test passes. This creates a sequential pipeline: Lint → Test → Build.

actions/cache@v4 — This action caches pip downloaded packages between workflow runs. The first run downloads all packages normally. Subsequent runs reuse the cache, significantly speeding up the Install dependencies step. The cache key is based on requirements.txt — if you change dependencies, the cache is invalidated and rebuilt.

`${{ github.sha }}` — This is a GitHub Actions expression. github.sha is a context variable that contains the commit SHA that triggered the workflow. We use it to tag the Docker image with a unique identifier.

Pipeline visualization:



7. Managing Secrets and Environment Variables

7.1 Why Never Hardcode Secrets

Our config.py file contains database credentials (host, username, password). In a real application, these are sensitive values that should NEVER be committed to your repository. Anyone with access to your repo could see them.

```
# config.py - Notice the default values
class Config:
    MYSQL_HOST = os.environ.get('MYSQL_HOST', 'localhost')
    MYSQL_USER = os.environ.get('MYSQL_USER', 'root')
    MYSQL_PASSWORD = os.environ.get('MYSQL_PASSWORD', 'admin') # ⚠ Default password!
    MYSQL_DB = os.environ.get('MYSQL_DB', 'sakila')
```

The good news is that our config.py already uses os.environ.get() with defaults. In CI, we can override these using GitHub Secrets — encrypted environment variables stored securely by GitHub.

7.2 Adding Secrets in GitHub

Step 1: Go to your repository on GitHub.

Step 2: Click on "Settings" (you need admin access to the repository).

Step 3: In the left sidebar, expand "Secrets and variables" and click on "Actions".

Step 4: Click the "New repository secret" button.

Step 5: Add your secrets one by one:

Secret Name	Example Value	Purpose
MYSQL_HOST	db.example.com	Database server address
MYSQL_USER	app_user	Database username
MYSQL_PASSWORD	str0ngP@ssw0rd!	Database password
MYSQL_DB	sakila	Database name

 **Warning:** Once a secret is saved, you cannot view its value again — you can only update or delete it. Make sure you save your secrets somewhere safe (like a password manager).

7.3 Using Secrets in Workflows

To use secrets in your workflow, reference them with the `${{ secrets.SECRET_NAME }}` syntax:

steps:

```
- name: Run tests with database
  env:
    MYSQL_HOST: ${ secrets.MYSQL_HOST }
    MYSQL_USER: ${ secrets.MYSQL_USER }
    MYSQL_PASSWORD: ${ secrets.MYSQL_PASSWORD }
    MYSQL_DB: ${ secrets.MYSQL_DB }
  run: |
    pytest tests/ -v
```

 **Note:** GitHub automatically masks secret values in workflow logs. If a secret value appears in any log output, it is replaced with *** to prevent accidental exposure.

Security best practices for secrets:

- Never print secrets in workflow logs (avoid echo \$SECRET)
- Use secrets only in the steps that need them (not globally)
- Rotate secrets periodically
- Use the principle of least privilege — give CI only the permissions it needs
- Never hardcode secrets in workflow files or source code

8. Working with Workflow Triggers

GitHub Actions supports many different types of events that can trigger workflows. Understanding triggers is essential for setting up effective CI pipelines.

8.1 Push and Pull Request Triggers

These are the most common triggers for CI:

```
on:
  push:
    branches: [ main, develop, 'release/**' ]
  pull_request:
    branches: [ main ]
```

- `push`: Triggers when commits are pushed to the specified branches.
- `pull_request`: Triggers when a PR is opened, updated (new commits pushed), or reopened against the specified branches.
- `'release/**'` uses a glob pattern to match any branch starting with `'release/'` (e.g., `release/v1.0`, `release/v2.0`).

8.2 Manual Triggers (`workflow_dispatch`)

Sometimes you want to trigger a workflow manually — for example, to re-run a deployment or run a one-off task:

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Target environment'
        required: true
        default: 'staging'
        type: choice
        options:
          - staging
          - production
      debug_enabled:
        description: 'Enable debug logging'
        required: false
        type: boolean
        default: false
```

With `workflow_dispatch`, a "Run workflow" button appears on the Actions tab. The inputs section defines form fields that the user can fill in before triggering.

8.3 Scheduled Triggers (cron)

You can run workflows on a schedule using cron syntax:

```
on:
  schedule:
    - cron: '0 6 * * 1-5'    # Run at 6:00 AM UTC, Monday through Friday
```

Cron syntax reference:

```

|_____ minute (0 - 59)
| |_____ hour (0 - 23)
| | |_____ day of the month (1 - 31)
| | | |_____ month (1 - 12)
| | | | |_____ day of the week (0 - 6, Sunday = 0)
| | | | |
* * * * *
```

Common examples:

Cron Expression	Meaning
'0 0 * * *'	Every day at midnight UTC
'30 8 * * 1'	Every Monday at 8:30 AM UTC
'0 */6 * * *'	Every 6 hours
'0 9 1 * *'	First day of every month at 9 AM UTC

8.4 Branch and Path Filters

You can filter triggers to only run when specific files or paths change:

```
on:
  push:
    branches: [ main ]
    paths:
      - '**.py'          # Only trigger if Python files changed
      - 'templates/**'  # Or if template files changed
      - 'requirements.txt' # Or if dependencies changed
```

```
paths-ignore:
- '**.md'           # Do NOT trigger for documentation changes
- '.gitignore'      # Do NOT trigger for gitignore changes
```

 **Note:** You cannot use both "paths" and "paths-ignore" together — use one or the other. "paths" is an allowlist (only trigger for these), while "paths-ignore" is a blocklist (trigger for everything except these).

9. Using Pre-Built Actions from the Marketplace

9.1 What Are Actions?

Actions are reusable components that encapsulate common CI/CD tasks. Instead of writing shell commands for everything, you can use pre-built actions that handle complex tasks in a single step. Think of them as plugins or extensions for your workflow.

Actions are referenced using the format:

```
uses: owner/action-name@version
```

For example:

```
uses: actions/checkout@v4      # GitHub official checkout action, version 4
uses: actions/setup-python@v5   # GitHub official Python setup, version 5
uses: docker/build-push-action@v5 # Docker's build and push action
```

9.2 Commonly Used Actions

Action	Purpose
actions/checkout@v4	Checks out your repository code onto the runner. Required in almost every workflow.
actions/setup-python@v5	Installs and configures a specific Python version on the runner.
actions/setup-node@v4	Installs and configures a specific Node.js version.
actions/cache@v4	Caches dependencies to speed up workflow runs.
actions/upload-artifact@v4	Uploads files (logs, reports, builds) as workflow artifacts.
actions/download-artifact@v4	Downloads artifacts uploaded by previous jobs.
docker/build-push-action@v5	Builds and optionally pushes Docker images.
docker/login-action@v3	Logs into a Docker registry (Docker Hub, GitHub Container Registry, etc.).

You can browse all available actions at the GitHub Marketplace:

<https://github.com/marketplace?type=actions>

9.3 Pinning Action Versions

There are three ways to specify an action version:

1. Major version tag (recommended): uses: actions/checkout@v4

This automatically gets the latest v4.x.x release. Good balance of stability and updates.

2. Full commit SHA (most secure): uses:

actions/checkout@8ade135a41bc03ea155e62e844d188df1ea18608

Pins to an exact commit. Cannot be changed after publishing. Best for security-sensitive workflows.

3. Branch name (least stable): uses: actions/checkout@main

Points to the latest code on the branch. Can change at any time. Not recommended for production workflows.

 **Tip:** For most projects, using major version tags (@v4) is the best approach. It provides stability while still receiving bug fixes and minor improvements.

10. Branch Protection and CI Enforcement

Branch protection rules prevent direct pushes to important branches (like main) and require that CI checks pass before code can be merged. This is how you enforce that all code goes through CI before reaching production.

10.1 Setting Up Branch Protection Rules

Step 1: Go to your repository on GitHub.

Step 2: Click "Settings" → "Branches" in the left sidebar.

Step 3: Under "Branch protection rules", click "Add branch protection rule" (or "Add classic branch protection rule").

Step 4: In "Branch name pattern", type: main

Step 5: Configure the following settings:

☒ **Require a pull request before merging:** This prevents anyone from pushing directly to the main branch. All changes must go through a pull request.

☒ **Require approvals (set to 1):** At least one team member must review and approve the pull request before it can be merged.

☒ **Require status checks to pass before merging:** This is the key setting for CI enforcement. It requires that specified CI checks pass before the PR can be merged.

☒ **Require branches to be up to date before merging:** Ensures the PR branch is up to date with the base branch, preventing merge conflicts.

10.2 Requiring Status Checks to Pass

Step 6: After enabling "Require status checks to pass before merging":

22. A search box appears. Type the name of your CI jobs (e.g., "Code Linting", "Run Tests", "Build Docker Image").

23. Select each job you want to require. These must match the job names in your workflow file.

24. Click "Create" (or "Save changes") at the bottom of the page.

Now, when someone creates a pull request:

- The CI pipeline runs automatically.
- The PR page shows the status of each required check.
- The "Merge" button is disabled (greyed out) until all required checks pass.
- If a check fails, the developer must push a fix and wait for CI to pass again.

 **Note:** Branch protection rules are only available for public repositories (free) or private repositories on paid GitHub plans (Pro, Team, Enterprise).

11. Adding a CI Status Badge to Your README

A CI status badge is a small image in your README that shows the current status of your CI pipeline (passing or failing). It provides instant visibility into the health of your project.

Step 1: Add the following markdown to your README.md:

```
# Sakila Flask Application

![CI Pipeline](https://github.com/YOUR-USERNAME/YOUR-REPO/actions/workflows/ci.yml/badge.svg)

... rest of your README ...
```

Step 2: Replace YOUR-USERNAME and YOUR-REPO with your actual GitHub username and repository name.

You can also get the badge URL from the GitHub UI:

25. Go to the Actions tab in your repository.
26. Click on a workflow name (e.g., "CI Pipeline") in the left sidebar.
27. Click the "..." (three dots) button in the top right.
28. Select "Create status badge".
29. Copy the generated markdown and paste it into your README.md.

The badge will display one of these statuses:

- "passing" (green) — The latest run of the workflow succeeded.
- "failing" (red) — The latest run of the workflow failed.
- "no status" (grey) — The workflow has never run or is currently running.

12. Troubleshooting Common Issues

Issue: Workflow does not trigger

- Verify the workflow file is in the correct path: `.github/workflows/ci.yml`
- Check that the YAML file is valid (no indentation errors). YAML is very sensitive to spaces.
- Ensure the trigger event matches your action (e.g., pushing to a branch listed in the 'on' section).
- Check the Actions tab for any error messages about invalid workflow files.
- Make sure the `.github/workflows/` directory is committed and pushed (not gitignored).

Issue: YAML syntax error

- YAML uses spaces for indentation — NEVER use tabs.
- Use a YAML validator (search for 'YAML lint online') to check your file.
- Ensure consistent indentation: 2 spaces per level is the standard.
- Colons in values must be quoted: `name: 'Value: with colon'`
- Special characters (`#`, `:`, `-`, `{`, `}`) in values should be quoted.

Issue: Module not found / Import error

- Ensure `requirements.txt` includes all necessary dependencies.
- Check that `pip install -r requirements.txt` runs before your test/lint step.
- If using a package not in `requirements.txt`, add it.
- Verify Python version matches what your code expects.

Issue: Tests pass locally but fail in CI

- CI runners start with a clean environment — no local databases, no environment variables.
- Check if tests depend on a database connection. If so, either mock the database or set up a service container.
- Check file paths — Linux (CI runner) uses forward slashes, Windows uses backslashes.
- Check for environment-specific behavior (timezone, locale, OS-specific commands).

Issue: Docker build fails in CI

- Run `'docker build .'` locally first to reproduce the issue.
- Check that all required files are not in `.dockerignore`.
- Ensure the Dockerfile is in the repository root (or adjust the context path).
- Check for platform-specific issues (your local machine may be ARM/Mac, CI runner is x86 Linux).

Issue: Workflow takes too long

- Add dependency caching (actions/cache) to avoid reinstalling packages every run.
- Use specific path triggers to avoid running CI for irrelevant changes (e.g., docs-only changes).
- Consider splitting into parallel jobs instead of sequential ones.
- Review if all steps are necessary for every run.

13. Best Practices Summary

- 1. Keep workflows fast:** Optimize for speed. Use caching, parallel jobs, and only run what is necessary. A slow CI pipeline discourages developers from pushing frequently.
- 2. Fail fast:** Put quick checks (linting) before slow checks (integration tests, Docker builds). This gives developers faster feedback on simple issues.
- 3. Use branch protection:** Require CI to pass before merging to main. This prevents broken code from reaching your production branch.
- 4. Pin action versions:** Use specific version tags (e.g., @v4) rather than @main or @latest. This prevents unexpected breakages when an action is updated.
- 5. Keep secrets secret:** Never hardcode passwords, API keys, or tokens in workflow files or source code. Always use GitHub Secrets.
- 6. Write meaningful test names:** Test names should describe what they verify. 'test_film_route_returns_200' is better than 'test_1'.
- 7. Monitor CI costs:** Keep an eye on your Actions minutes usage, especially for private repositories. Unnecessary workflow runs consume your quota.
- 8. Use path filters:** Avoid running the full CI pipeline when only documentation or non-code files change.
- 9. Review workflow files in PRs:** Treat workflow changes like code changes — review them carefully, as they have access to your secrets and can run arbitrary code.
- 10. Start simple, iterate:** Begin with a basic lint + test pipeline. Add complexity (Docker, deployments, matrix builds) only when needed.

14. Glossary of Terms

Term	Definition
Action	A reusable, pre-packaged task used as a step in a workflow (e.g., actions/checkout).
Artifact	A file or set of files produced during a workflow run (e.g., test reports, build outputs).
Branch Protection	Rules that restrict who can push to a branch and what checks must pass.
Cache	Stored dependencies from a previous run, reused to speed up subsequent runs.
CI (Continuous Integration)	The practice of automatically building and testing code on every change.
CD (Continuous Delivery)	The practice of keeping code always in a deployable state, with manual deployment.
CD (Continuous Deployment)	The practice of automatically deploying every change that passes CI.
Commit SHA	A unique 40-character hash identifying a specific Git commit.
Cron	A time-based scheduling system using a 5-field expression (minute, hour, day, month, weekday).
Docker Image	A lightweight, standalone package containing everything needed to run an application.
Event	An activity that triggers a workflow (e.g., push, pull_request, schedule).
Flake8	A Python linting tool that checks code against PEP 8 style guidelines.
GitHub Marketplace	A directory of pre-built Actions available for use in workflows.
Job	A set of steps that execute on the same runner within a workflow.
Linting	Automated checking of source code for style issues and potential errors.
Pipeline	A series of automated stages (lint, test, build, deploy) that code passes through.
Pull Request (PR)	A request to merge changes from one branch into another, with review and CI checks.

PyTest	A popular Python testing framework used to write and run automated tests.
Runner	A virtual machine that executes workflow jobs. Can be GitHub-hosted or self-hosted.
Secret	An encrypted environment variable stored in GitHub, used for sensitive data like passwords.
Status Check	A CI result (pass/fail) that can be required before merging a pull request.
Step	An individual task within a job — either a shell command (run:) or a pre-built action (uses:).
Workflow	An automated process defined in a YAML file, triggered by events.
YAML	A human-readable data format used to write GitHub Actions workflow files.

— End of Tutorial —